INSTITUT SUPERIEUR D'INFORMATIQUE

ET DES TECHNIQUES DE COMMUNICATION – HAMMAM SOUSSE

المعهد العالي للإعلامية وتكنولوجيات الاتصال بسوسة

End-of-Studies Internship Report

Presented for obtaining the Bachelor degree in Computer Science
Spécialité : Informatique and Multimédia

CyberScope

Web Application Security Auditing System
Detecting and Analyzing Vulnerabilities
Through Intelligent Automation

ELABORATED BY
MOHAMED GUIZANI

Academic Supervisor: M.Mohamed Amine Ben Amar

Professional Supervisor: M.Yassine Lasoued

Host Company

EY

School Year : 2025/2026

Higher Institute of Computer Science and Communication Technologies ISITCOM Tel/Fax: +216 73 37 15 71 / +216 73 36 44 11

Dedications

First and foremost, I thank **Allah** for choosing me among thousands to face this trial, which has made me more patient, wiser, and stronger. Despite the back injury that has challenged me for more than a year and a half, during which I could barely sit to work, all praise be to Allah for His guidance and strength.

To my beloved family, who replaced my physical pain with their unwavering support, love, and care:

To **Anissa, my mother**, whose devotion knew no bounds—checking on my condition and illness every single day throughout this difficult period. Your constant care has been my healing.

To **Hassen, my father**, who dedicated himself entirely to ensuring my comfort, doing everything possible to ease my burden. Your sacrifices have not gone unnoticed.

To **Iskander**, who became my backbone and eternal supporter when my own failed me. You carried me through the darkest moments.

To **Sabri**, from whom I drew power and strength when mine was depleted. Your presence gave me energy to continue.

To **Naktel**, the youngest among us, who brought joy and happiness into days filled with pain. Your light brightened my path.

Every line of this report was written through immense pain and is dedicated to all of you. Your love transformed my struggle into strength, and your faith in me made this achievement possible.

Alhamdulillahi rabbil alameen.

Acknowledgments

I would like to express my deepest gratitude to all those who have contributed to the completion of this thesis.

First and foremost, I extend my heartfelt appreciation to **M. Yassine Lassoed**, my professional supervisor, who believed in me and in this project from the very beginning. Despite the challenges faced during this internship, he provided me with essential guidance and constant support, enabling me to bring this project to life. His encouragement and trust in my abilities have been invaluable throughout this journey.

I would also like to sincerely thank **M. Mohamed Amine Ben Amar**, my academic supervisor, for his continuous support and valuable advice. Throughout this challenging period, he offered insightful feedback, motivation, and understanding, which greatly contributed to the successful completion of this work.

I am equally grateful to all the individuals who supported me, directly or indirectly, during the realization of this project. Their encouragement and assistance played a significant role in achieving this accomplishment.

Thank you for making this achievement possible.

Table of Contents

Dedications **i**

Acknowledgements **ii**

Table of Contents **iii**

List of Figures **vi**

List of Tables **vii**

General Introduction **1**

1 General Presentation of the Project **2**

1.1 Introduction 2

1.2 Presentation of the Host Organization 2

1.3 Project Context and Problem Statement 3

1.4 Overview of Web Application Security 3

1.4.1 OWASP Top 10 and Modern Threat Landscape 4

1.4.2 Limitations of Traditional Vulnerability Scanners 4

1.5 Study of Existing Solutions 5

1.5.1 Nikto 5

1.5.2 OWASP ZAP 5

1.5.3 Burp Suite 5

1.5.4 Comparative Analysis 6

1.6 Critique of Existing Solutions 7

1.7 Proposed Solution: CyberScope 7

1.8 Adopted Development Methodology 8

1.9 Conclusion 8

2 System Analysis and Requirements Specification **9**

2.1 Introduction 9

2.2 Stakeholder Identification 9

2.3 System Overview 9

2.4 Functional Requirements 10

2.4.1 Scan Initiation and Target Submission 10
2.4.2 Intelligent Tool Orchestration 11
2.4.3 Dynamic Application Crawling (BrowserAgent) 11
2.4.4 Vulnerability Detection and Analysis 11
2.4.5 Results Aggregation and Visualization 11
2.5 Non-Functional Requirements 12
2.5.1 Performance and Scalability 12
2.5.2 Security Requirements 12
2.5.3 Usability 12
2.6 Use Case Diagram 12
2.7 Conclusion 13

3 System Design and Architecture 14
3.1 Introduction 14
3.2 Global System Architecture 14
3.3 Detailed Architecture 14
3.3.1 Frontend Architecture (React) 15
3.3.2 Backend Architecture (Node.js) 15
3.3.3 Scanning Engine (Python / HexStrike) 15
3.4 Key Components 16
3.4.1 BrowserAgent 16
3.4.2 Orchestration Engine 16
3.4.3 Vulnerability Analyzer 16
3.5 Data Flow and Processing Pipeline 17
3.6 Scrum Alignment of the Architecture 18
3.7 Conclusion 19

4 System Implementation (Sprint-Based Development) 20
4.1 Introduction 20
4.2 Sprint 4 — Testing and Experimental Validation 20
4.2.1 Test Environment and Methodology 20
4.2.2 Case Study: OWASP Juice Shop 20
4.2.3 Case Study: Altoro Mutual 20
4.2.4 Sprint Outcome 20
4.3 Results Discussion 20
4.3.1 True Positives vs False Positives 20
4.3.2 Detection Coverage 20
4.3.3 System Limitations 20

4.4 Comparative Evaluation with Existing Tools 20
4.5 Proposed Enhancements 21
4.6 Future Work 21
4.7 Conclusion 21

5 Validation and Results **22**
5.1 Introduction 22
5.2 Comparative Evaluation with Existing Tools 22
5.3 Identified Limitations 22
5.4 Proposed Enhancements 22
5.5 Future Research Directions 22
5.6 Conclusion 22

References **23**

General Conclusion **25**

List of Figures

1.1 EY Tunisia 3
1.2 Nikto 5
1.3 OWASP ZAP 5
1.4 Burp Suite 6

2.1 Global Use Case Diagram of CyberScope 13

3.1 Global Architecture of CyberScope 14
3.2 CyberScope Scan Processing Pipeline 18

List of Tables

1.1 Comparative analysis of existing solutions 6
1.2 Synthesis of limitations of existing solutions 7

3.1 Main Architectural Layers of CyberScope 15
3.2 Key Components and Responsibilities 17
3.3 Architecture Alignment with Scrum Sprints 19

General Introduction

In recent years, the rapid evolution of web technologies has significantly transformed the development and deployment of modern applications. Web applications have become increasingly dynamic, interactive, and complex, relying heavily on client-side rendering and continuous communication with backend services. While these advancements have enhanced user experience, they have also expanded the attack surface and introduced new security challenges.

Web application security has become a critical concern for organizations. Vulnerabilities such as SQL injection, cross-site scripting (XSS), and authentication flaws remain among the most exploited attack vectors, as highlighted by the OWASP Top 10. These vulnerabilities can lead to severe consequences, including data breaches, financial losses, and reputational damage.

To mitigate these risks, various security scanning tools have been developed, including solutions such as Nikto, OWASP ZAP, and Burp Suite. Although these tools provide valuable capabilities, they exhibit significant limitations. In particular, traditional scanners struggle to effectively analyze modern Single Page Applications (SPAs), where content is dynamically generated through JavaScript. Furthermore, they often produce redundant or low-priority findings, making vulnerability analysis more complex and less efficient.

In this context, there is a growing need for more intelligent and adaptive security auditing systems capable of handling modern web architectures while improving detection accuracy and result relevance.

To address these challenges, this project introduces **CyberScope**, an automated web application security auditing system designed to detect and analyze vulnerabilities through intelligent orchestration and advanced automation techniques. The proposed system integrates multiple security tools within a unified pipeline and enhances their capabilities through a browser-based crawling agent capable of exploring dynamic applications. In addition, an AI-driven correlation engine is employed to normalize, deduplicate, and prioritize detected vulnerabilities, providing more meaningful and actionable insights.

The objective of this work is to design and implement a comprehensive and efficient security auditing platform that improves vulnerability detection coverage and adapts to the complexity of modern web environments.

Chapter 1

General Presentation of the Project

1.1 Introduction

This chapter focuses on the general context of the project. It presents the background of the work, the security challenges faced by modern web applications, the limitations of existing solutions, and the motivation behind the proposed system, **CyberScope**.

1.2 Presentation of the Host Organization

Ernst & Young (EY) [1] is one of the world's leading professional services firms, recognized globally for its expertise in assurance, consulting, strategy, tax, and transaction services. Founded in 1989 through the merger of Ernst & Whinney and Arthur Young, EY operates in more than 150 countries and employs hundreds of thousands of professionals worldwide.

In Tunisia, EY plays a key role in supporting organizations in their digital transformation and technological innovation. The firm assists both public and private institutions in addressing complex business challenges by providing tailored consulting services and advanced technical solutions.

EY Tunisia offers a wide range of services, including IT consulting, cybersecurity, risk management, and digital innovation. In particular, the firm contributes to strengthening information systems security by identifying vulnerabilities and implementing robust protection mechanisms.

Through its global network and local expertise, EY promotes best practices in cybersecurity and technological development. Its commitment to quality and innovation makes it an ideal environment for the development of advanced solutions such as automated web security auditing systems.

Figure 1.1: EY Tunisia

1.3 Project Context and Problem Statement

With the increasing reliance on web applications in critical domains such as finance, e-commerce, and healthcare, ensuring their security has become a major challenge. Modern applications are more dynamic and complex, which significantly increases their exposure to various cyber threats such as SQL injection, cross-site scripting, and authentication vulnerabilities.

Despite the availability of several security tools, existing solutions often fail to effectively analyze modern web applications, especially those based on dynamic architectures. These tools may miss critical vulnerabilities, generate redundant results, or require significant manual intervention, which reduces their efficiency and reliability.

In this context, the main problem addressed in this project is the lack of an intelligent and automated solution capable of efficiently auditing modern web applications while improving vulnerability detection accuracy and reducing false positives.

The objective of this project is therefore to design and develop **CyberScope**, an advanced web application security auditing system that enhances vulnerability detection through intelligent orchestration, automation, and improved analysis of security findings.

1.4 Overview of Web Application Security

Web application security refers to the set of measures and practices implemented to protect web applications from vulnerabilities and cyber threats. As web applications become increasingly central to business operations and data management, ensuring their security is essential to preserve the confidentiality, integrity, and availability of information.

Modern web applications are exposed to a wide range of attacks that exploit weaknesses in their design, implementation, or configuration. These attacks may target input validation, authentication mechanisms, session management, or communication

between client and server. As a result, a single vulnerability can compromise the entire system and lead to severe consequences such as data breaches, financial loss, or reputational damage.

To mitigate these risks, security must be integrated throughout the entire development lifecycle. This includes secure coding practices, regular vulnerability assessments, and the use of automated security tools capable of identifying potential weaknesses before they are exploited.

In this context, standards such as the OWASP Top 10 provide valuable guidance by identifying the most critical web application security risks [2]. These references help developers and security professionals prioritize their efforts and implement effective protection strategies.

1.4.1 OWASP Top 10 and Modern Threat Landscape

The OWASP Top 10 is a widely recognized standard that identifies the most critical security risks affecting web applications [2]. It provides a prioritized list of vulnerabilities such as injection attacks, broken authentication, sensitive data exposure, and cross-site scripting.

These vulnerabilities represent the most common attack vectors exploited by attackers in real-world scenarios. As web technologies evolve, the threat landscape continues to expand, with modern applications introducing new risks related to client-side execution, APIs, and complex system interactions.

By highlighting the most significant threats, the OWASP Top 10 helps developers and security professionals focus on the most impactful vulnerabilities and adopt effective mitigation strategies.

1.4.2 Limitations of Traditional Vulnerability Scanners

Traditional vulnerability scanners are widely used to detect security flaws in web applications through automated analysis and predefined signatures. While these tools are effective for identifying known vulnerabilities, they present several limitations when applied to modern web environments.

In particular, they often struggle to analyze dynamic applications, especially those based on client-side rendering and asynchronous communication. As a result, important attack surfaces may remain unexplored. Additionally, these tools may generate a high number of false positives and redundant findings, which complicates the analysis process and reduces their overall efficiency.

These limitations highlight the need for more advanced and intelligent approaches to web application security auditing.

1.5 Study of Existing Solutions

Several tools have been developed to identify vulnerabilities in web applications and assist security professionals in detecting potential threats. These tools provide different approaches to vulnerability scanning, ranging from simple automated checks to advanced interactive analysis. Among the most widely used solutions are Nikto, OWASP ZAP, and Burp Suite.

1.5.1 Nikto

Nikto is an open-source web server scanner designed to detect known vulnerabilities, outdated software, and misconfigurations in web servers [4]. It performs comprehensive checks against a large database of known issues and is commonly used for quick security assessments.

However, Nikto is limited to server-side analysis and does not effectively handle modern dynamic web applications, which reduces its applicability in complex environments.

Figure 1.2: Nikto

1.5.2 OWASP ZAP

OWASP ZAP is a widely used open-source security testing tool that provides automated scanning as well as manual testing capabilities [3]. It is capable of detecting a variety of vulnerabilities such as injection flaws and security misconfigurations.

Despite its flexibility, ZAP may require proper configuration and user expertise to achieve optimal results, and it can produce a significant number of false positives in complex applications.

Figure 1.3: OWASP ZAP

1.5.3 Burp Suite

Burp Suite is a professional web security testing platform offering a comprehensive set of tools for vulnerability detection and exploitation [5]. It allows detailed

analysis of HTTP requests and responses and supports advanced manual testing techniques.

While powerful, Burp Suite is more complex to use and often requires significant manual intervention, which can make the testing process time-consuming.

Figure 1.4: Burp Suite

1.5.4 Comparative Analysis

To better evaluate the strengths and limitations of the studied tools, a comparative analysis is presented below. Although Nikto, OWASP ZAP, and Burp Suite are widely used in web application security testing, each of them addresses only part of the overall auditing process.

Nikto is well suited for rapid server-side checks, OWASP ZAP provides a balanced approach between automation and usability, and Burp Suite offers advanced manual testing capabilities. However, none of these tools fully combines intelligent automation, deep dynamic exploration, and effective reduction of noisy findings.

Table 1.1: Comparative analysis of existing solutions

Criteria	**Nikto**	**OWASP ZAP**	**Burp Suite**
Type	Open-source server scanner	Open-source proxy and scanner	Professional security testing suite
Main approach	Signature-based scanning	Automated and manual testing	Advanced manual and automated testing
Dynamic application support	Limited	Partial	Good
Ease of use	Easy	Moderate	Complex
False positives	High	Medium	Low to medium
Manual testing support	No	Yes	Advanced
Automation level	High	High	Medium
Best suited for	Quick server checks	General web security testing	Deep vulnerability analysis

This comparison shows that existing tools provide useful but incomplete capabilities. Their limitations in terms of automation, adaptability, and dynamic analysis

justify the need for a more integrated and intelligent solution such as **CyberScope**.

1.6 Critique of Existing Solutions

Although existing tools such as Nikto, OWASP ZAP, and Burp Suite provide valuable support for vulnerability detection, they still present several limitations when applied to modern web applications.

These limitations are mainly related to the lack of support for dynamic applications, the generation of false positives, and the need for manual intervention. In addition, the absence of intelligent correlation between detected vulnerabilities reduces the clarity and efficiency of the analysis process.

Table 1.2: Synthesis of limitations of existing solutions

Limitation	**Description**	**Impact**
Limited support for dynamic applications	Difficulty handling JavaScript-based applications	Incomplete vulnerability detection
High false positives	Redundant or irrelevant findings	Time-consuming analysis
Lack of intelligent correlation	No prioritization of vulnerabilities	Reduced clarity of results
Manual intervention required	Dependence on user expertise	Low scalability
Fragmented tool usage	Multiple tools used separately	Complex workflow

These limitations highlight the need for a more advanced solution capable of combining automation, accuracy, and adaptability.

1.7 Proposed Solution: CyberScope

To overcome the limitations identified in existing solutions, this project proposes **CyberScope**, an intelligent web application security auditing system.

CyberScope is based on a unified architecture that integrates multiple security tools into a single automated pipeline. It introduces a browser-based crawling mechanism capable of exploring modern dynamic applications and interacting with client-side components.

In addition, the system incorporates an intelligent analysis layer responsible for normalizing, deduplicating, and prioritizing vulnerabilities. This significantly reduces noise and improves the relevance of the results.

By combining automation, dynamic exploration, and intelligent processing, Cy-

berScope aims to provide a more efficient and scalable approach to web application security auditing.

1.8 Adopted Development Methodology

To ensure an efficient and flexible development process, the Scrum methodology was adopted for the realization of this project. Scrum is an agile framework that promotes iterative development, continuous feedback, and adaptability.

The project was organized into multiple sprints, each focusing on specific objectives such as backend development, tool integration, frontend implementation, and system refinement. This iterative approach allowed progressive improvement of the system while ensuring regular validation of functionalities.

Throughout the development process, tasks were planned, executed, and evaluated in each sprint. This facilitated rapid issue resolution, continuous enhancement, and better management of project complexity.

The adoption of Scrum contributed to improving productivity, organization, and adaptability, which were essential for the successful development of **CyberScope**.

1.9 Conclusion

This chapter introduced the general context of the project by highlighting the importance of web application security and the challenges posed by modern dynamic applications. It also presented a study of existing solutions and identified their main limitations.

Based on this analysis, the need for a more advanced and intelligent approach was established, leading to the proposal of **CyberScope**. The next chapters will focus on the system analysis, design, and implementation of the proposed solution.

Chapter 2

System Analysis and Requirements Specification

2.1 Introduction

This chapter presents the analysis of the proposed system, **CyberScope**. It identifies the main stakeholders involved in the platform and defines the functional and non-functional requirements that guide its development.

It also provides an overview of the system and highlights its core functionalities, including intelligent tool orchestration, dynamic application crawling, and automated vulnerability analysis, which form the foundation for the system design presented in the next chapter.

2.2 Stakeholder Identification

The development and use of **CyberScope** involve several stakeholders, each interacting with the system according to specific roles.

The primary stakeholder is the **Security Analyst**, who represents the main user of the platform. This actor is responsible for submitting target URLs, initiating automated scans, and analyzing the results through the system dashboard. The analyst reviews detected vulnerabilities, examines their severity levels, and analyzes the evidence generated during the scanning process.

The **System Administrator** is responsible for deploying and maintaining the platform. This includes managing system configurations, ensuring the availability of integrated scanning tools, including the Python-based scanning engine and its integrated tools, and supervising backend services to guarantee proper system operation.

Academic and professional supervisors are considered indirect stakeholders. They are involved in evaluating the project, validating its objectives, and ensuring that the developed solution meets both technical and academic requirements.

Finally, the **target application owner** represents an external stakeholder. Although they do not directly interact with the system, their web applications are analyzed by CyberScope. They benefit from the identification of vulnerabilities and the actionable results produced by the platform.

2.3 System Overview

CyberScope is an intelligent web application security auditing platform designed to automate the detection and analysis of vulnerabilities in modern web environments. It

aims to provide a unified solution that combines multiple security tools with advanced analysis capabilities.

The system is structured around three main components: a web-based user interface, an orchestration backend, and a scanning engine. The user interacts with the platform through a dashboard that allows the submission of target URLs, the initiation of scans, and the visualization of results.

When a scan is launched, the backend coordinates the execution of multiple security tools through an automated pipeline. The scanning engine performs the analysis by combining traditional vulnerability scanning techniques with dynamic exploration of web applications. The system also generates a **target profile** characterizing the attack surface, technology stack, and exposed services of the analyzed application.

The collected results are then processed, aggregated, and presented to the user in a structured and readable format. This global architecture enables CyberScope to handle complex and dynamic applications while improving the accuracy and relevance of detected vulnerabilities.

2.4 Functional Requirements

The functional requirements define the core services that CyberScope must provide to ensure efficient and automated web application security auditing.

The system must allow users to initiate security scans on target web applications and manage the entire scanning lifecycle. It should support the integration of multiple security tools and coordinate their execution through an automated and intelligent pipeline.

CyberScope must also be capable of analyzing modern web applications by incorporating dynamic exploration mechanisms, enabling deeper coverage of client-side behavior and hidden attack surfaces. In addition, the system must detect and analyze vulnerabilities by combining the outputs of different tools and presenting them in a structured manner.

Finally, the platform must provide a clear and intuitive visualization of scan results, allowing users to easily interpret findings, assess their severity, and make informed decisions regarding security improvements.

2.4.1 Scan Initiation and Target Submission

The system must allow the user to submit a target web application in the form of a URL and initiate a security scan. This functionality represents the starting point of the auditing process.

Once a target is provided, the system must validate the input and automatically trigger the scanning workflow. The platform should manage the execution of the scan without requiring manual intervention, ensuring a seamless and efficient user

experience.

In addition, the system must support the management of multiple scan requests, allowing users to monitor ongoing scans and access previously executed ones.

2.4.2 Intelligent Tool Orchestration

The system must ensure the coordinated execution of multiple security tools within a unified scanning pipeline. This functionality enables CyberScope to perform comprehensive vulnerability assessments by combining the strengths of different tools.

The platform should automatically select and execute the appropriate tools based on the characteristics of the target application. It must manage the execution flow, ensuring that tools operate efficiently and without unnecessary redundancy.

By orchestrating multiple tools in an intelligent manner, the system improves detection coverage and enhances the overall quality of the security analysis.

2.4.3 Dynamic Application Crawling (BrowserAgent)

The system must support the exploration of modern web applications through a dynamic crawling mechanism. This functionality is essential for analyzing Single Page Applications and client-side interactions that are not accessible through traditional scanning techniques.

To achieve this, the platform integrates a browser-based agent capable of interacting with web pages, triggering user actions, and capturing underlying network requests. This allows the system to uncover hidden endpoints and improve the detection of vulnerabilities in dynamic environments.

2.4.4 Vulnerability Detection and Analysis

The system must be capable of detecting security vulnerabilities by analyzing the outputs generated by multiple integrated tools. It should identify common web application vulnerabilities such as SQL injection, cross-site scripting (XSS), path traversal, and security misconfigurations.

In addition, the platform must analyze the detected findings to provide meaningful insights, allowing users to understand the nature and potential impact of each vulnerability.

2.4.5 Results Aggregation and Visualization

The system must process and aggregate scan results obtained from different tools to eliminate redundancy and improve clarity. This includes organizing findings and applying deduplication mechanisms such as hash-based comparison to reduce duplicated results.

The results must be presented through a structured and intuitive interface, enabling users to easily interpret the findings and make informed decisions regarding

security improvements.

2.5 Non-Functional Requirements

In addition to functional requirements, CyberScope must satisfy several non-functional requirements to ensure reliability, efficiency, and usability.

2.5.1 Performance and Scalability

The system should provide acceptable response times during scan execution and result processing. Scan initiation should remain within acceptable response times (typically within a few seconds), ensuring a responsive user experience.

It must also be scalable to handle multiple scan requests and support the analysis of large and complex web applications.

2.5.2 Security Requirements

The platform must ensure secure access control and protect sensitive data related to scan results and system operations. It should implement secure authentication mechanisms, such as token-based authentication, and safeguard communications between system components.

2.5.3 Usability

The user interface must be intuitive and easy to use for security analysts. The visualization of results should be clear and well-structured, facilitating the interpretation of findings and improving user experience.

2.6 Use Case Diagram

The use case diagram illustrates the main interactions between the primary actor, the security analyst, and the CyberScope system. It highlights the core functionalities of the platform, including submitting targets, launching scans, and analyzing results.

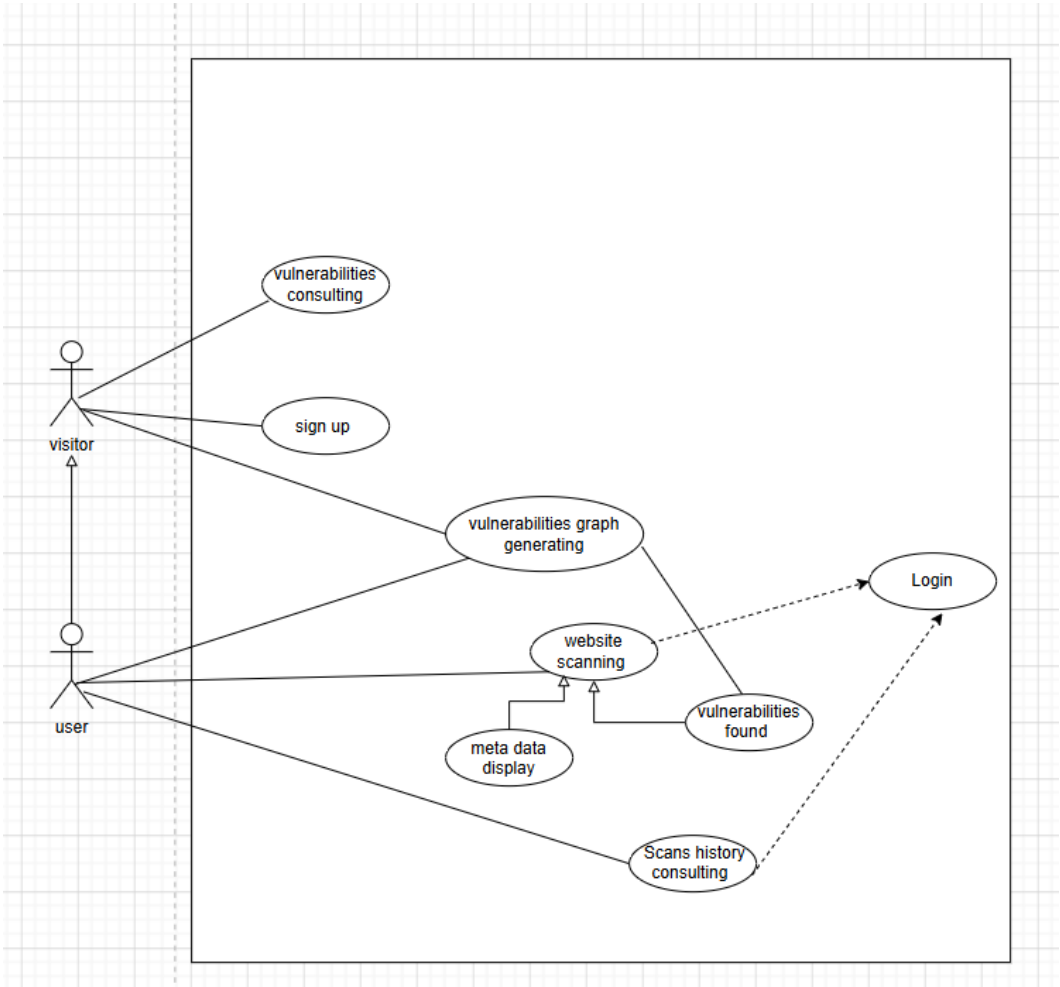

Figure 2.1: Global Use Case Diagram of CyberScope

2.7 Conclusion

This chapter presented the analysis of the CyberScope system by identifying its stakeholders and defining both functional and non-functional requirements. It established a clear understanding of the system's objectives and functionalities, providing a solid foundation for the design and architecture presented in the next chapter.

Chapter 3

System Design and Architecture

3.1 Introduction

This chapter presents the design and architecture of the proposed solution, **CyberScope**. It describes the global structure of the system, its main components, and the data flow followed during the scanning process.

The objective of this chapter is to define how the different parts of the platform interact before moving to the implementation phase. The design choices presented here also prepare the sprint-based implementation described in the next chapter.

3.2 Global System Architecture

CyberScope follows a multi-layer architecture composed of three main parts: the frontend dashboard, the Node.js orchestration backend, and the Python-based scanning engine. This separation improves modularity, maintainability, and scalability.

The frontend provides the user interface through which the security analyst submits targets, launches scans, and views results. The backend manages authentication, scan lifecycle, database operations, and communication with the scanning engine. The Python engine is responsible for target profiling, tool orchestration, dynamic crawling, and vulnerability discovery.

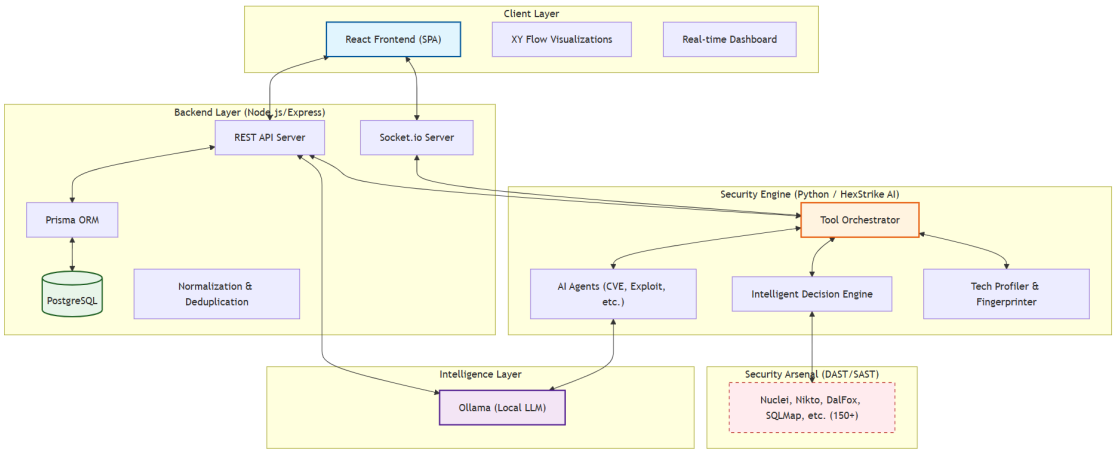

Figure 3.1: Global Architecture of CyberScope

3.3 Detailed Architecture

The detailed architecture of CyberScope is organized around three complementary layers. Each layer has a specific responsibility and communicates with the others through well-defined interfaces.

3.3.1 Frontend Architecture (React)

The frontend of CyberScope is developed using React and TypeScript. It provides the main interaction point between the user and the platform. Through the dashboard, the security analyst can submit a target URL, monitor scan execution, view target profile information, and analyze the detected vulnerabilities.

The frontend includes several main pages such as the authentication pages, the scanner interface, the scan history page, and the scan details page. These interfaces are designed to provide a clear visualization of results and support efficient decision-making.

3.3.2 Backend Architecture (Node.js)

The backend is implemented using Node.js and Express. It acts as the orchestration layer between the frontend, the database, and the Python scanning engine. Its responsibilities include receiving scan requests, managing authentication, storing scan data, and exposing API endpoints to the frontend.

The backend also includes services responsible for scan management, communication with the HexStrike engine, vulnerability matching, normalization, and attack path processing. This modular organization makes the system easier to maintain and extend.

3.3.3 Scanning Engine (Python / HexStrike)

The scanning engine is implemented as a Python-based server. It represents the core security analysis layer of CyberScope. Its role is to analyze the submitted target, build a target profile, select appropriate tools, and execute the scanning pipeline.

The engine integrates several security tools such as Nuclei, SQLMap, Nikto, Gobuster, Dalfox, Arjun, Nmap, and other reconnaissance or vulnerability detection modules. It also includes the BrowserAgent component, which improves analysis coverage for modern dynamic web applications.

Table 3.1: Main Architectural Layers of CyberScope

Layer	**Role**	**Main Technologies**
Frontend Dashboard	User interaction, scan submission, result visualization	React, TypeScript, Vite
Orchestration Backend	API management, authentication, scan lifecycle, database access	Node.js, Express, PostgreSQL
Scanning Engine	Target profiling, tool selection, crawling, vulnerability detection	Python, Flask, Selenium, mitmproxy, security tools

3.4 Key Components

The architecture of CyberScope is based on several key components that provide the main intelligence and automation capabilities of the system.

3.4.1 BrowserAgent

The BrowserAgent is one of the most important components of CyberScope. It is designed to analyze modern web applications that rely heavily on JavaScript and client-side rendering.

Unlike traditional crawlers, the BrowserAgent uses a real browser environment to interact with the application. It can trigger user actions, navigate through dynamic pages, and capture network requests generated by the frontend. This allows CyberScope to discover hidden API endpoints and parameters that static scanners may miss.

The BrowserAgent also supports traffic interception through mitmproxy, enabling the system to observe requests produced during navigation and use them as input for deeper vulnerability testing.

3.4.2 Orchestration Engine

The orchestration engine is responsible for coordinating the execution of multiple security tools. Instead of launching all tools randomly, the system analyzes the target and selects the appropriate tools according to its characteristics.

This component improves scanning efficiency by avoiding unnecessary execution and organizing the workflow between reconnaissance, crawling, vulnerability detection, and result collection.

3.4.3 Vulnerability Analyzer

The vulnerability analyzer processes the raw findings produced by different tools. Its role is to normalize the results, reduce duplicates, assign severity levels, and match findings against known vulnerability definitions.

This component improves the readability and usefulness of the final results. Instead of presenting raw outputs directly to the user, CyberScope provides structured findings with severity, evidence, and contextual information.

Table 3.2: Key Components and Responsibilities

Component	**Responsibility**
BrowserAgent	Explores dynamic applications, interacts with client-side elements, and captures API requests.
Orchestration Engine	Selects and coordinates security tools according to the target profile.
Vulnerability Analyzer	Normalizes, deduplicates, prioritizes, and structures detected findings.
Target Profile Module	Characterizes the target attack surface, technologies, services, and risk indicators.
Dashboard	Displays scans, findings, target metadata, and severity distribution to the user.

3.5 Data Flow and Processing Pipeline

The CyberScope scanning process follows a structured pipeline. The user first submits a target URL from the frontend dashboard. The backend receives the request and creates a scan record before forwarding the target to the Python scanning engine.

The scanning engine analyzes the target and generates a target profile. Based on this profile, the orchestration engine selects the appropriate security tools and launches them. The raw results are then returned to the backend, where they are normalized, deduplicated, matched with known vulnerability definitions, and stored in the database.

Finally, the frontend retrieves the processed results and displays them in a clear format, including vulnerability details, severity levels, evidence, and scan diagnostics.

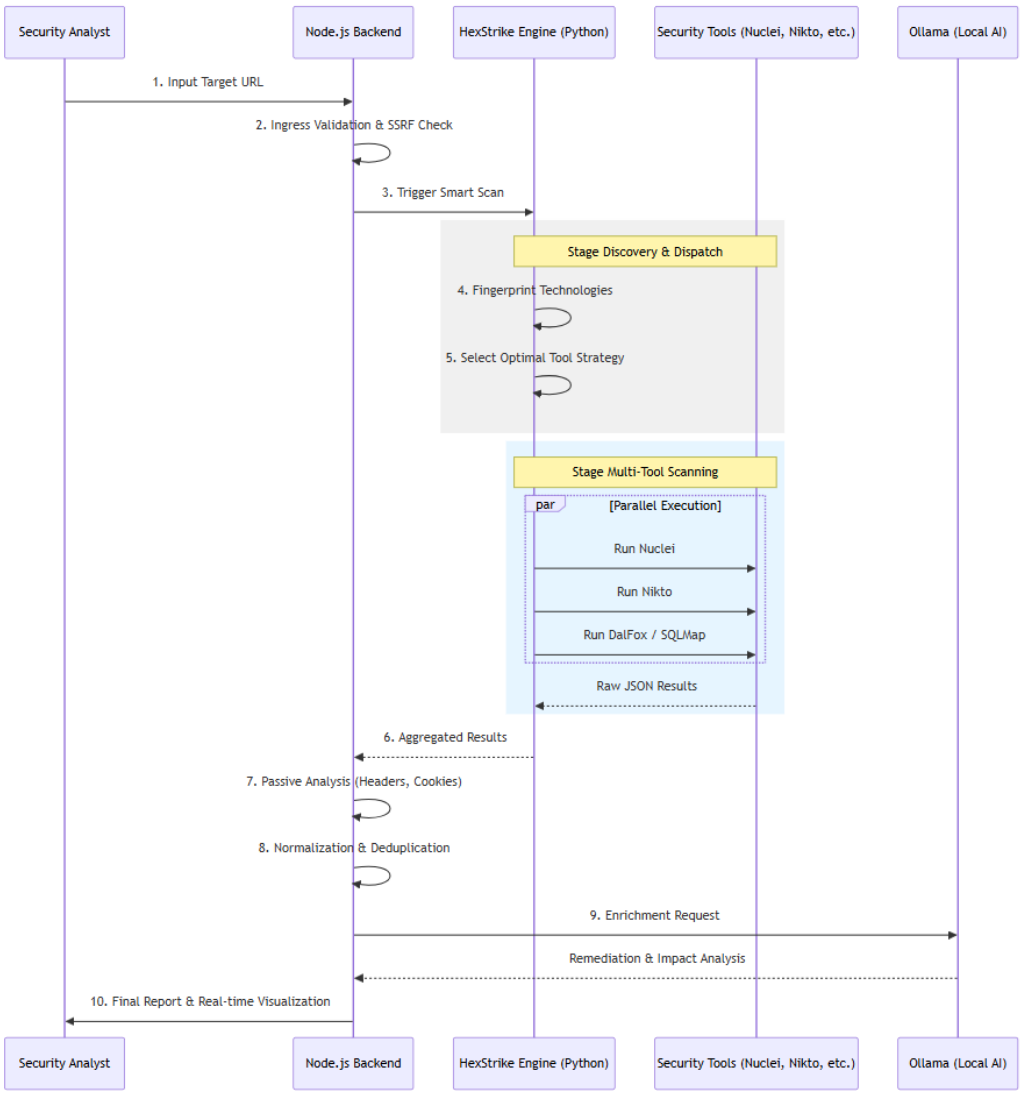

Figure 3.2: CyberScope Scan Processing Pipeline

3.6 Scrum Alignment of the Architecture

The architecture of CyberScope was designed to support the sprint-based implementation process. Each sprint focuses on a specific layer or functional block of the system, allowing progressive development and validation.

Table 3.3: Architecture Alignment with Scrum Sprints

Sprint	**Architectural Focus**	**Main Deliverables**
Sprint 1	Backend infrastructure and database layer	API structure, authentication, PostgreSQL schema, Python engine connection
Sprint 2	Scanning engine and security pipeline	Tool integration, target profiling, intelligent orchestration, normalization engine
Sprint 3	Frontend dashboard and user interaction	Scanner page, scan history, results display, target profile visualization
Sprint 4	Validation and testing	Case studies, scan evaluation, result analysis, system limitations

3.7 Conclusion

This chapter presented the design and architecture of CyberScope. It described the global architecture, the main technical layers, the key components, and the scan processing pipeline.

The proposed architecture provides a solid foundation for an intelligent and automated web application security auditing system. The next chapter will focus on the sprint-based implementation of these architectural components.

Chapter 4

System Implementation (Sprint-Based Development)

4.1 Introduction

This chapter presents the validation of CyberScope and discusses the main results obtained during experimental testing. It also analyzes the system's effectiveness, limitations, and possible future improvements.

4.2 Sprint 4 — Testing and Experimental Validation

Content will be added here.

4.2.1 Test Environment and Methodology

Content will be added here.

4.2.2 Case Study: OWASP Juice Shop

Content will be added here.

4.2.3 Case Study: Altoro Mutual

Content will be added here.

4.2.4 Sprint Outcome

Content will be added here.

4.3 Results Discussion

Content will be added here.

4.3.1 True Positives vs False Positives

Content will be added here.

4.3.2 Detection Coverage

Content will be added here.

4.3.3 System Limitations

Content will be added here.

4.4 Comparative Evaluation with Existing Tools

Content will be added here.

4.5 Proposed Enhancements

Content will be added here.

4.6 Future Work

Content will be added here.

4.7 Conclusion

This chapter presented the experimental validation of CyberScope and discussed the obtained results. It also identified the main limitations of the system and outlined possible future improvements.

Chapter 5

Validation and Results

5.1 Introduction

This chapter summarizes the final results of the project and discusses the main limitations, possible improvements, and future research directions for CyberScope.

5.2 Comparative Evaluation with Existing Tools

Content will be added here.

5.3 Identified Limitations

Content will be added here.

5.4 Proposed Enhancements

Content will be added here.

5.5 Future Research Directions

Content will be added here.

5.6 Conclusion

This chapter highlighted the final outcomes of the project and outlined possible future evolutions of the proposed system.

References

Bibliography

[1] Ernst & Young (EY), *EY Tunisia Official Website*. Available at: https://www.ey.com/fr_ca/locations/tunisia

[2] OWASP Foundation, *OWASP Top 10 - Web Application Security Risks*, 2021.

[3] OWASP ZAP, https://www.zaproxy.org/

[4] Nikto Web Server Scanner, https://cirt.net/Nikto2

[5] PortSwigger, *Burp Suite*, https://portswigger.net

General Conclusion

Conclusion